Kai Sheng

734-802-8898 | shengkai866@gmail.com | Ann Arbor, MI | linkedin.com/in/kai-sheng

EDUCATION

University of Michigan – Ann Arbor Sep 2026 – Dec 2027
Computer Science and Engineering (Master of Science), College of Engineering *Ann Arbor, MI*

University of Michigan – Ann Arbor Sep 2022 – May 2026
Computer Science (Bachelor of Science), College of Literature, Science, and the Arts *GPA: 3.85 / 4.00*

INTERNSHIP EXPERIENCE

Meituan May 2026 – Aug 2026
Software Development Engineer, Data Platform *Beijing, China*

- Built an enterprise-IM agent for the food-delivery flow team that automates the full data-dev pipeline — PM requirement intake → metric-definition alignment → validation → ETL development → testing → review — serving PMs and data developers.
- Lowered PM↔dev communication overhead: PMs can start from a vague idea; the agent runs multi-turn elicitation to progressively surface requirement background, scope, and metric dimensions, then auto-generates the requirement doc, creates an issue in multica, and spins up a cross-functional group with business and dev — replacing scheduled alignment meetings, cutting the requirement-handoff cycle from ~2 days to ~1 day, and reducing rework from ambiguous requirements.
- Automated ETL development so developers only review code: the agent lands code changes and new-table DDLs directly on the XT platform, runs platform tests, and verifies action SQL end-to-end; developers now review rather than write ETL from scratch.
- Fused BM25 and semantic-vector (HNSW) hybrid search for first-stage recall, then a resident cross-encoder reranker for the final top-10; validated knowledge-base quality with an LLM-driven eval pipeline (auto-question → answer → grade → merge): on a 189-question sampled set (63 sources × 3), retrieval routing matched 100%; a Pairwise contradiction scan across all 2,300 segments surfaced 0 conflicts.
- Verified metric definitions against data assets: the agent auto-traverses Hive table schemas, lineage, and upstream/downstream dependencies to check whether a metric definition is realizable and matches the PM-signed definition.
- Orchestrated a three-phase relay pipeline — orchestrator → plan-and-execute + ReAct leader → kb-search retrieval — with a resident cross-encoder reranker. Enforced human-in-the-loop gates and a red-line rule: confirmation gates act as hard sync points where agents may only change implementation paths, never the PM-signed definition; reduced out-of-spec changes from ~12% to ~6%.

University of Michigan – Ann Arbor Sep 2025 – Present
Research Assistant – Software Engineer · GLKB (Genomic Literature Knowledge Base) *glkb.org*

- The team built a large-scale biomedical knowledge graph covering 33M PubMed articles, 263M biomedical terms, and 627M relationships; designed and exposed RESTful tool APIs (entity retrieval, literature search, relation/ontology queries, subgraph extraction) for AI Agents to power GraphRAG-based Q&A with citation-backed traceability.
- Implemented a multi-layer semantic retrieval pipeline: ANN recall over 100M sentence embeddings via Faiss, followed by Neo4j graph traversal to join entity relations and context; fused vector and graph knowledge as LLM context to support GraphRAG Q&A.
- Developed a streaming LLM chat interface over SSE that integrates graph retrieval results into generative responses with inline literature citations; introduced KV-cache-based interaction in a multi-agent architecture, reducing end-to-end response latency from 1,596 ms to 445 ms.
- Designed the graph database layer on Neo4j supporting 55M nodes and 627M relationships; applied a Router → Schema → Service → Neo4jManager layered architecture with async/await for high-concurrency I/O.
- Deployed services on AWS with a compute/API-split architecture: ran Agent inference as a long-lived network service on EC2 (compute-intensive, kept resident to avoid reload overhead); used Lambda as the serverless entry point, where each request is first authenticated and rate-limited against PostgreSQL before being forwarded to the Agent service; Lambda autoscales per request, absorbing most high-concurrency load at the edge.

PROJECT EXPERIENCE

Distributed System Search Engine Aug 2025 – Dec 2025
Two-stage retrieval + Learning-to-Rank *Python, Hadoop, React, BERT*

- Built an end-to-end distributed document search engine from scratch spanning text preprocessing, inverted-index construction, two-stage retrieval, and Learning-to-Rank (L2R), with a React frontend and a Flask REST API.
- Built an offline data pipeline with Python and Hadoop MapReduce to process a 200K-document corpus and generate segmented inverted indexes.
- Used BERT/Transformer to generate Top-10 latent user queries per document as document expansion, added to the index to improve semantic recall and matching.
- Engineered multi-signal ranking features including TF-IDF, BM25, Bi-encoder and Cross-encoder relevance features, plus PageRank/HITS graph-structure features.
- Trained a Learning-to-Rank (L2R) model on 13,555 labeled samples; adopted a two-stage architecture with BM25 + Bi-encoder first-stage recall and L2R top-100 re-ranking.
- Achieved NDCG@10 ≈ 0.51 and MAP@10 ≈ 0.36 on the Top-10 ranking task, significantly improving ranking quality over TF-IDF/BM25 baselines.

SUMMARY

Languages: Python, SQL, JavaScript, Java, C++, R, HTML/CSS
AI / ML: LangChain, LangGraph, BERT/Transformers, PyTorch, scikit-learn, Pandas, NumPy
Frameworks: Flask, FastAPI, React, Spark, Hadoop
Databases: Neo4j, PostgreSQL, MySQL, MongoDB, Hive, Oracle
Cloud: AWS (Lambda, EC2, Amplify)